AN INTERNSHIP REPORT

on

"GROUND CONTROL SYSTEM FOR UAVs"

Submitted to

KIIT Deemed to be University

In Partial Fulfillment of the Requirement for the Award of

BACHELOR'S DEGREE IN
Electronics and Computer Science Engineering

BY

AVILASHA GOSWAMI 2230162

UNDER THE GUIDANCE OF
Subhrajeet Jena
Product Manager, Kristellar Aerospace Pvt. Ltd.

School of Electronics Engineering

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

BHUBANESWAR, ODISHA - 751024

November 2025

A PROJECT REPORT

on

"GROUND CONTROL SYSTEM FOR UAVs"

Submitted to

KIIT Deemed to be University

In Partial Fulfillment of the Requirement for the Award of

BACHELOR'S DEGREE IN

Electronics and Computer Science Engineering

BY

AVILASHA GOSWAMI 2230162

UNDER THE GUIDANCE OF

SUBHRAJEET JENA

Product Manager, Kristellar Aerospace Pvt. Ltd.

School of Electronics Engineering

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

BHUBANESWAR, ODISHA -751024

November 2025

KIIT Deemed to be University

School of Electronics Engineering
Bhubaneswar, ODISHA 751024

CERTIFICATE

This is certify that the project entitled

"GROUND CONTROL SYSTEM FOR UAVs "

submitted by

AVILASHA GOSWAMI Roll number: 2230162

is a record of bonafide work carried out by them, in the partial fulfillment of the requirement for the award of Degree of Bachelor of Engineering (Electronics and Computer Science) at KIIT Deemed to be university, Bhubaneswar. This work is done during year 2025-2026, under our guidance.

Date: 02 /12 /2025

(Subhrajeet Jena)
Project Mentor
Kristellar Aerospace Pvt. Ltd

Acknowledgements

I am profoundly grateful to Subhrajeet Jena of Kristellar Aerospace Pvt. Ltd. for his expert guidance and continuous encouragement throughout to see that this project rights its target since its commencement to its completion. 1st August 2025 - Present.

AVILASHA GOSWAMI

ABSTRACT

A ground control system (GCS) is a specialized software and hardware platform designed to allow operators to monitor, control, and manage unmanned aerial vehicles (UAVs) remotely. GCS forms the interface between human operators and the vehicles, enabling the transmission of command and telemetry data over communication links such as radio, Wi-Fi, or USB. Essential GCS functionalities include mission planning (such as waypoints and autonomous routes), real-time flight status monitoring, failure and health management, live video streaming, payload control, and remote configuration of onboard devices. For drone missions, the GCS is crucial as it enables safe and efficient multi-vehicle coordination, pre-flight setup (firmware flashing, calibration), and post-flight data analysis. Advanced workflows include automated deployment and software packaging for cross-environment compatibility, comprehensive dependency management for robust execution on diverse systems, and multi-vehicle simulation support for swarm defense scenarios or research applications.

This work focused on developing a rebranded, portable Linux GCS system, including custom methodical steps for robust packaging (AppImage and shell-based installer solutions), streamlining hardware connectivity for diverse autopilots. This involved manual aggregation of all runtime libraries, plugins, and resources to produce a completely portable, self-contained executable. Extensive automation was developed to install, configure, and launch the GCS in varied environments with dependency detection and error handling.

Keywords:

Ground Control System (GCS)
UAV Integration
Multi-Vehicle Simulation
Linux Packaging
Telemetry Communication

Contents

1	Introduction			1
2	Basic Concepts			2
	2.1	Unmanned Aerial Vehicles (UAVs)		2
	2.2	Ground Control System (GCS)		2
	2.3	Flight Controllers		2
	2.4	Linux Deployment Tools and Libraries		2
	2.5	AppImage Packaging		2
3	Problem Statement / Requirement Specifications			3
	3.1	Project Planning		3
	3.2	Project Analysis		3-4
	3.3	System Design		4
		3.3.1	Design Constraints	4
		3.3.2	System Architecture / Block Diagram	4
4	Implementation			5
	4.1	Methodology / Proposal		5
	4.2	Testing / Verification Plan		6
	4.3	Result Analysis / Screenshots		7
5	Standard Adopted			8
	5.1	Design Standards		8
	5.2	Coding Standards		9
	5.3	Testing Standards		9
6	Conclusion and Future Scope			10
	6.1	Conclusion		10
	6.2	Future Scope		10
References				11
Individual Contribution				12

List of Figures

4.3.1 IMAGE CAPTION GCS Interface 6

4.3.2 IMAGE CAPTION Gazebo Simulator for Quadcopter 7

4.3.3 IMAGE CAPTION CubePilot Cube Orange+ Flight Controller 7

Chapter 1

Introduction

The rapid expansion of unmanned aerial systems (UAS) across defence, surveillance, industrial and research domains has created a growing demand for reliable, secure and platform-independent Ground Control Station (GCS) software. Existing solutions such as QGroundControl provide strong core functionality, but they also present several limitations when deployed in customised, organisation-specific, or resource-constrained environments. In many cases, these tools are not directly optimised for defence-oriented workflows, custom rebranding, offline deployment, multi-vehicle coordination, or cross-platform portability.

A major need identified in the current ecosystem is the absence of a self-contained, portable, and reconfigurable GCS application that can be deployed seamlessly on any Linux system without dependency conflicts. At the same time, modern applications require advanced simulation capabilities, such as Software-In-The-Loop (SITL), Hardware-In-The-Loop (HITL), and multi-vehicle swarm simulations using PX4 and Gazebo. However, most ready-to-use GCS packages do not provide scripts, automation, or tuning support for launching multiple concurrent vehicles, resolving MAVLink communication congestion, or integrating automated takeoff and formation logic.

This work addresses these gaps by designing, customising, rebranding, deploying, and validating a fully operational Ground Control System named “RiGEL”. The work includes: configuring a complete Qt 6.8.3 build environment, resolving plugin-level issues with QtQuick and QML modules, modifying build systems for full rebranding, and developing two portable distribution approaches; AppImage packaging and a universal self-extracting Linux installer. The work also attempts to integrate real CubePilot hardware to validate firmware updates, GPS setup, battery calibration, and sensor configuration. Through these efforts, the RiGEL system becomes an end-to-end deployable GCS solution capable of supporting autonomous missions, swarm coordination, and field-ready hardware interfacing.

Chapter 2

Basic Concepts

This section explains the fundamental concepts, tools, and technologies that form the foundation of this project.

2.1 Unmanned Aerial Vehicles (UAVs):

Unmanned Aerial Vehicles (UAVs), commonly known as drones, are aircraft operated without an onboard human pilot. They rely on embedded flight controllers, sensor inputs, communication systems and control algorithms to maintain stability, navigate environments, and execute missions. UAVs are widely used in surveillance, mapping, logistics, defence operations and research.

2.2 Ground Control Systems (GCS):

A Ground Control System is the operator interface used to monitor, configure, and control UAVs. A GCS typically provides real-time telemetry display, mission planning tools, map visualisation, sensor readings, firmware updates and command execution. It serves as the communication bridge between the pilot and the UAV, enabling tasks such as takeoff, landing, mode switching, waypoint-based missions and parameter configuration.

2.3 Flight Controllers:

A flight controller is the onboard computing unit responsible for stabilisation, navigation, sensor fusion, and executing control algorithms. It receives data from sensors such as accelerometers, gyroscopes, compasses, GPS modules, barometers and cameras. Using this information, the flight controller continuously adjusts motor outputs to maintain stable flight.

2.4 Linux Deployment Tools and Libraries:

Deploying complex applications in Linux requires understanding components such as:

- **Shared Libraries (.so files)** – Dynamically loaded at runtime to support UI, sensors, communication and graphics.
- **Qt Framework** – Provides GUI components, QML modules, multimedia, networking and hardware abstraction.
- **Plugin Systems** – Used to load UI controls, maps, themes and runtime extensions.

2.5 AppImage Packaging:

AppImage is a Linux packaging format that bundles an application and all its dependencies into a single portable executable file. It eliminates the need for installation, avoids library conflicts and enables distribution across different Linux systems. AppImage combines an AppDir structure, metadata files, launcher scripts and embedded libraries into one self-contained file.

Chapter 3

Problem Statement / Requirement Specifications

Modern UAV operations increasingly demand reliable, secure, and portable ground control capabilities that can be deployed across varied environments, hardware platforms and network restrictions. However, existing UAV control systems often face challenges such as dependency conflicts on Linux platforms, limited portability, hardware interface issues with flight controllers, and a lack of customizable deployment options. The core problem addressed in this project is the need for a stable, customizable, portable, and simulation-ready Ground Control System environment that can work across diverse Linux systems, support real UAV hardware, enable multi-vehicle testing, and provide a clean deployment mechanism without reliance on system-installed libraries.

3.1 Project Planning

Project execution began by identifying the essential components required for a complete UAV control environment:

3.1.1 Identifying Required Features:

- Support for mission planning, waypoint management and telemetry monitoring.
- Compatibility with autopilot firmware (PX4/ArduPilot).
- Stable Linux build environment for UAV control software.
- Deployment as a self-contained AppImage and as a universal shell-based installer.
- Ability to run on multiple Linux systems without installation errors.

3.1.2 Planning Development Workflow:

- Set up the Ubuntu development environment.
- Install required build tools, compilers and libraries.
- Compile the application from source.
- Package the final application for distribution (AppImage + Installer).

3.2 Project Analysis

Once the requirements were conceptualized, a detailed analysis was carried out to identify design considerations, ambiguities and constraints.

3.2.1 Identifying Potential Issues:

- Linux library mismatches causing plugin failures.
- QtQuick and QML modules failing to load on target systems.
- GPS modules not detected due to incorrect protocol selection or misconfigured CAN parameters.
- Failsafe triggers such as low battery warnings or EKF calibration errors.

3.2.2 Feasibilty Assessment:

- Ensured that the system can support both simulation and hardware testing.
- Verified that AppImage packaging is feasible with correct bundling of Qt libraries.

3.2.3 Risk Evaluation:

- Risk of segmentation faults caused by missing UI plugins.
- System crashes due to Qt version mismatch.

3.3 System Design

3.3.1 Design Constraints:

The project required a well-defined environment to support building, executing, simulating and testing UAV control operations. The main design constraints were:

- Operating System: Ubuntu Linux (22.04 / 24.04)
- Programming Tools: GCC/G++, CMake, Ninja, Python3
- Frameworks: Qt (version 6.x), QML, QtQuick Controls
- Packaging Tools: linuxdeployqt, AppImageKit, shell-based installer scripts
- UAV Flight Controller: Cube series (e.g., Cube Orange+/Black)
- GPS Module: CAN-based modules (Here3/Here3+)
- Connectivity: USB interfaces, CAN ports, telemetry radios
- Test Environment: Host computer with sufficient CPU/RAM for multi-SITL

3.3.2 System Architecture:

The overall system architecture integrates simulation tools, hardware devices, communication protocols and deployment systems into a unified workflow.

1. User Interface Layer: Mission planning, Map visualization, Telemetry monitoring, Parameter configuration, Sensor and status display.

2. Application Core Layer: Mission Control logic, communication with MAVLink, user command handling, vehicle state management.

3. Autopilot Interaction Layer: Firmware flashing and bootloader access, parameter reading and writing, sensor calibration workflows.

4. Communication Layer: MAVLink over UDP (for SITL), MAVLink over serial, CAN bus for GPS and peripherals, USB/Serial device discovery.

5. Simulation Layer: SITL interfaces for single or multiple UAVs, Gazebo environment interaction.

6. Hardware Layer: Flight controllers (Cube series), GPS modules (DroneCAN), Sensors (IMU, magnetometer, barometer), Servo and actuator interfaces.

7. Deployment Layer: Appimage packaging, self-extracting installer, Library bundling and environment setup, cross-machine portability enhancements.

Chapter 4

Implementation

This chapter describes the actual work performed during the development of the project, including the methodology followed, the tools and algorithms adopted, the testing and verification processes, the analysis of results and screenshots, and the quality assurance measures taken to ensure correctness and reliability.

4.1 Methodology

The implementation methodology followed a structured, multi-stage development process to ensure that all components; software, simulation, hardware and deployment were systematically integrated.

Step1: Environment Setup and Dependency Installation:

- Prepared a Linux-based development environment (Ubuntu).
- Installed compilers, build tools, CMake, Ninja, Python and essential system libraries.
- Configured the Qt framework, QML modules, QtQuick Controls, multimedia plugins and required dependencies.

Step2: Application Compilation and Runtime Debugging:
Built the application from source using CMake and GCC.
Resolved errors related to missing plugins, QtQuick modules, GStreamer multimedia components and system libraries.
Ensured that the compiled program successfully processed telemetry, user inputs, mapping, mission planning and sensor data.

Step3: Deployment Engineering:
Approaches:
AppImage Packaging

- Created AppDir structure, embedded libraries, plugins and QML components.
- Used linuxdeployqt and AppImageKit to generate a standalone executable.

Self-Extracting Installer (Shell-based)

- Developed an installer script containing an embedded compressed AppDir archive.
- Script extracts files, configures environment variables and creates a launcher command.
- Enabled universal portability without requiring AppImage support.

Step4: Simulation Integration:

- Installed PX4 Autopilot and set up SITL simulation.
- Integrated Gazebo simulator for physics-based 3D environments.
- Validated telemetry flow, mission planning functionality and command responsiveness.

Step5: Hardware Interfacing and Configuration:

- Interfaced Cube-series flight controllers over USB and serial.
- Performed firmware flashing, bootloader access, and parameter synchronization.
- Faced failsafe warnings (battery voltage, EKF initialization, GPS configuration).

4.2 Testing OR Verification Plan

A structured verification plan was developed to ensure that each functional component of the system performed correctly. The table below outlines key test cases executed during the project.

Table 1: Test Plan Layout

Test ID	Test Case Title	Test Condition	System Behavior	Expected Result
T01	Application Execution test	Run built application on Ubuntu machine	Loads GUI, initializes modules, starts without errors	Application launches successfully without crashes
T02	Dependency Resolution Test	Execute AppImage on a fresh Linux system	System checks for libraries and loads bundled versions	AppImage runs without requiring external packages
T03	Installer Extraction test	Run self-extracting installer script	Extracts AppDir, sets paths, creates launcher	Application installs locally and runs via launcher command
T04	Firmware Flashing Test	Connect flight controller and trigger firmware upload	Enters bootloader, flashes firmware, resets successfully	Firmware updated without device errors
T05	Sensor Calibration Test	Perform IMU/magnetometer/barometer calibration	Calibration routines execute successfully	Sensor offsets saved and accepted

4.3 Result Analysis OR Screenshots

In this subsection, the output of the experiment or study in terms of some graphs, plots must be presented.

4.3.1 Deployment Results:

- AppImage executed successfully on a clean Linux system without missing dependencies.
- Self-extracting installer performed full installation and created a functional launcher command.

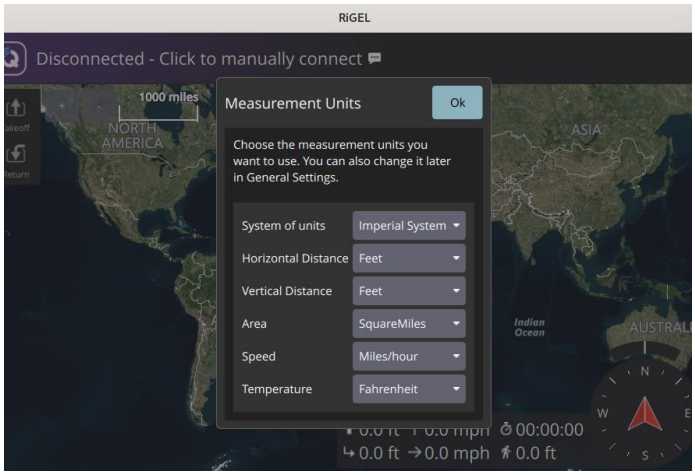

Fig 4.3.1: GCS Interface

4.3.2 Simulation Results:

- Successful launch of single-vehicle SITL with stable telemetry.
- Correct waypoint execution and mission planning through the interface.

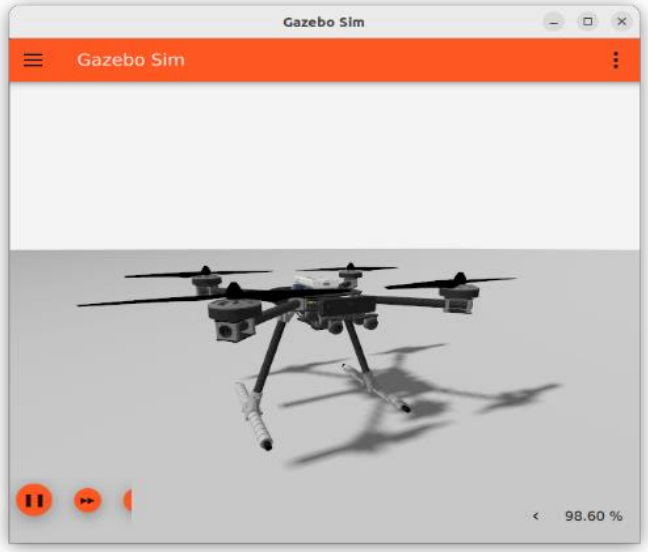

Fig 4.3.2: Gazebo Simulator for Quadcopter

4.3.3 Hardware Results:

- Flight controller correctly detected via USB and serial interfaces.
- Firmware flashing completed using the bootloader mechanism.
- Battery voltage and power module settings validated.

Fig 4.3.3: CubePilot Cube Orange+ Flight Controller

Chapter 5

Standards Adopted

5.1 Design Standards

During system design and planning, several established engineering standards and modelling practices were adopted to ensure clarity, maintainability and correctness of the system architecture.

5.1.1 System Design and Modelling Practices:

- UML (Unified Modeling Language):
    - Used conceptual diagrams such as use-case descriptions, activity flows and block-level representations.
    - Facilitated understanding of interactions between simulation modules, hardware components and communication layers.
- Modular Design Methodology:
  System was divided into individual modules such as communication layer, deployment layer, simulation layer and hardware abstraction layer.
- Layered Architecture Approach:
  Adopted to separate interface, logic, communication and hardware layers.
- Standards for Hardware Interfacing:
  Followed guidelines for UAV communication protocols such as MAVLink and CAN-bus standards for GPS modules and peripherals.

5.2 Coding Standards

Coding standards help maintain readability, consistency, and long-term maintainability of software systems. The following coding practices were adopted during the implementation:

- **Minimal and efficient code**
  Avoided unnecessary lines, redundant loops and repeated logic.
- **Clear and meaningful naming conventions**
  Variables, functions, modules and scripts were named based on their purpose (e.g., start_simulation.sh, install_rigel.sh).
- **Segmentation of code blocks**
  Related functions were grouped together for easy navigation and modification.
- **Proper indentation and formatting**
  Used 4-space indentation consistently to clearly represent structures such as loops, conditionals and function blocks.
- **Single responsibility principle**
  Ensured each function performed only one logical task (e.g., installer path setup, dependency checking, SITL vehicle launch, MAVSDK control actions).
- **Use of comments and documentation**
  Added inline comments, header comments and descriptive logs for critical operations, especially in shell scripts and automation routines.

- **Avoidance of long functions**
  Complex procedures were broken into smaller functions or separate scripts to improve clarity.
- **Error handling and safety checks**
  Verified USB and serial device availability before attempting connection, added checks for missing libraries in installer scripts, included safe-exit patterns for SITL multi-vehicle launch.
- **Version control discipline**
  Regular commits and logs were maintained to track changes during development.

5.3 Testing Standards

Testing and verification were conducted according to established quality assurance frameworks and internationally recognized standards. These served as guidelines to evaluate the completeness, correctness and reliability of the final system.

5.3.1 Standards Referenced:

- **IEEE 829 – Software Test Documentation Standard**
  Guided the preparation of test cases, expected results, and documentation of outcomes.
- **IEEE 1012 – Verification and Validation (V&V) Standard**
  Used to ensure the correctness of simulation behaviour, hardware interfacing and dependency resolution.
- **ISO/IEC/IEEE 29119 – Software Testing Standard**
  Provided a framework for test process stages such as planning, design, execution and reporting.
- **ISO/IEC 25040 – Quality Evaluation Process**
  Referenced during deployment verification to evaluate non-functional parameters such as portability and usability.

5.3.2 Practical Testing Guidelines Followed:

- Each functional block was tested independently (unit-style testing).
- SITL simulation tests verified UAV state transitions, telemetry flow and mission execution.
- Hardware-level tests validated GPS detection, firmware updates, battery failsafes and sensor calibration.
- Deployment tests ensured installer correctness, library bundling and cross-system portability.

Chapter 6

Conclusion & Future Scope

6.1 Conclusion

The project successfully achieved the development of a complete UAV control and simulation ecosystem, covering software compilation, simulation integration, deployment engineering, and hardware interfacing. A portable and fully functional ground control environment was designed and implemented, enabling mission planning, telemetry monitoring, autonomous flight execution, and multi-vehicle simulation. Through systematic debugging, rigorous dependency management and the creation of a universal installer, the system now operates reliably across multiple Linux platforms without external library conflicts.

Comprehensive simulation workflows were established using PX4 SITL and Gazebo, enabling safe testing of single and multi-UAV scenarios. Custom scripts were developed to automate multi-vehicle launches, reduce communication congestion and support swarm behaviour testing. The project also delivered a robust deployment mechanism through AppImage packaging and a self-extracting Linux installer, allowing the system to run seamlessly on diverse machines. The overall implementation ensured reliability, portability, scalability and usability, demonstrating a complete understanding of UAV software ecosystems, communication protocols, hardware components, and simulation frameworks.

6.2 Future Scope

While the project achieved its intended goals, several areas can be explored for further development and enhancement:

- **Advanced Swarm Intelligence**
  The current multi-vehicle simulation can be extended to incorporate formation control algorithms, inter-drone communication, collision avoidance strategies, distributed routing and autonomous coordination.
- **AI-Based Mission Planning**
  Machine learning techniques can be used to optimise waypoint placement, flight paths, energy consumption and real-time obstacle avoidance in dynamic environments.
- **Cross-Platform Deployment**
  The system can be extended to support Windows-based installers, macOS deployments and Mobile/tablet-based ground stations.
- **Enhanced User Interface and Data Visualisation**
  Future improvements can include customizable dashboards, graph-based telemetry analysis, 3D mission visualization and real-time data filtering for advanced user control.

References

[1] L. Meier et al., "MAVLink: Micro Air Vehicle communication protocol," *ArduPilot/MAVLink Documentation*, 2022. [Online]. Available: https://mavlink.io/
[2] PX4 Autopilot Development Team, "PX4 User Guide and Developer Documentation," 2024. [Online]. Available: https://docs.px4.io/
[3] Open Robotics, "Gazebo: Robot simulation environment," 2024. [Online]. Available: https://gazebosim.org/
[4] DroneCAN Working Group, "DroneCAN Protocol Specification," 2023. [Online]. Available: https://dronecan.github.io/
[5] The Qt Company, "Qt 6 Documentation: QML, QtQuick, QtQuick Controls," 2024. [Online]. Available: https://doc.qt.io/
[6] AppImage Project, "AppImage documentation: Portable Linux apps," 2024. [Online]. Available: https://appimage.org/

SAMPLE INDIVIDUAL CONTRIBUTION REPORT:

GROUND CONTROL SYSTEM FOR UAVs

Avilasha Goswami
Roll number: 2230162

Abstract: This project focuses on developing a portable and fully functional UAV control and simulation environment capable of operating across diverse Linux systems. The work includes creating a complete software deployment setup, integrating UAV autopilot firmware with simulation tools, supporting real-time hardware interfacing, and enabling multi-vehicle testing. The system provides mission planning, autonomous flight execution, and reliable communication capabilities for both simulated and real UAV platforms.

Individual contribution and findings: As a key contributor to this project, I was responsible for establishing the complete software, simulation, hardware, and deployment workflow required to build a reliable UAV control environment. My work began with setting up the Linux-based development ecosystem, which included installing compilers, libraries, build tools, Python utilities, and system dependencies required for UAV operations.

A major part of my contribution involved application compilation and debugging, where I identified the root causes behind segmentation faults, dependency failures, and version conflicts. I ensured that the program executed consistently on the development system by verifying module initialization, telemetry processing, sensor data flow, and mission-related features. I thoroughly analysed Linux shared libraries and manually reconstructed missing Qt modules when needed to achieve a stable runtime environment.

Another contribution was in deployment engineering, where I developed two independent distribution mechanisms:

1. A single-file AppImage executable, created by constructing the AppDir structure, bundling libraries, embedding QML modules, and invoking deployment tools.
2. A self-extracting universal installer, which I wrote entirely from scratch. It automatically extracts the application files, sets environment paths, checks for missing system packages, and creates a reusable launcher command.

I also implemented the complete simulation pipeline. This included installing PX4 Autopilot, configuring Software-in-the-Loop (SITL) simulation, and integrating the Gazebo 3D environment. I validated the flight behaviour of simulated vehicles by monitoring telemetry, executing mission commands, and testing autonomous takeoff and landing sequences.

In the hardware domain, I worked on Cube-series flight controller integration, addressing USB detection issues, serial port conflicts, and bootloader access. I performed firmware flashing, parameter synchronization, IMU and compass calibration, EKF validation, and battery configuration. I also configured DroneCAN-based GPS modules, corrected protocol conflicts, addressed missing GPS detection issues, and validated sensor status indicators.

Individual contribution to project report preparation: For the preparation of the project report, I contributed to drafting and finalizing the following chapters and sections:

- **Chapter1:** Introduction
- **Chapter2:** Basic Concepts
- **Chapter3:** Problem Statement, Requirement Specifications, System Design
- **Chapter4:** Implementation (Methodology, Testing, Result Analysis, Quality Assurance)
- **Chapter5:** Standards Adopted
- **Chapter6:** Conclusion and Future Scope
- **References Section**
- Formatting, structuring, editing, and ensuring consistency across the entire document
- Organizing screenshots, diagrams, system workflows, and technical explanations

I ensured that the report followed the required formatting guidelines, maintained technical clarity, and accurately represented the work completed during the project.

Individual contribution for project presentation and demonstration: During the demonstration phase, I was responsible for showcasing the technical components of the system, including the Software-in-the-Loop (SITL) setup, UAV takeoff using MAVSDK, and the functionality of the universal installer. I demonstrated how real hardware components such as the flight controller and GPS modules are connected, configured, calibrated, and verified.

Full Signature of Project Mentor: Full signature of the student:

Avilasha Goswami

…….